How I Used AI Tools, and Limitations / Technical Challenges

Project: Candidate Management Mini App (React + Express + PostgreSQL + Gemini)

1. How AI tools helped me during development

I used **Claude Code** as my AI coding assistant. My core work was analyzing the requirements, coming up with ideas and design decisions, breaking the work into smaller tasks, reviewing the code, testing, and taking responsibility for the final product. I read the requirements myself, defined the scope and feature priorities, came up with ideas for the architecture and the user experience, and then gave specific tasks to Claude Code. Claude Code helped implement the parts I assigned, explained technical choices and suggested ways to handle errors; I reviewed, adjusted and integrated the results into the product.

My workflow:

1. **Analyzing and deciding on my own:** I compared the specification with the time available, settled the frontend/backend structure, the data and the business flow, and wrote down my assumptions wherever the PDF was unclear.
2. **Assigning clearly scoped tasks:** I asked Claude Code to implement one small part at a time, following the architecture I had chosen, instead of letting the AI decide the whole product.
3. **Reading and owning the code:** I checked the changes, compared them with the project conventions, asked for explanations of anything unclear, and adjusted the code to fit the overall structure.
4. **Verifying the results myself:** I ran the application, called the API, checked the interface and ran the tests; when something failed, I reproduced the problem, looked for the cause and confirmed the fix by running it again.

Claude Code helped implement parts such as the backend modules (controller, service, Zod schema, TypeORM entity, migration, seed), the React interface, the Gemini integration on the backend, error handling, tests and documentation. I did not treat the AI's suggestions as correct by default: before accepting them I checked their behavior, security, data rules and fit with the requirements.

Decisions and directions I made myself:

- I chose TypeORM instead of Prisma as the project's technology decision, and I develop against PostgreSQL installed on my machine instead of depending on Docker.
- I prioritized finishing CRUD and the mandatory features first, and I separated client and server so that the API key is only used on the backend.
- I settled the data fields, the hiring statuses, how accent-insensitive Vietnamese search works, pagination and the choice of rows per page.
- I decided to support an English / Vietnamese / Korean interface and to keep API error messages in one consistent format.
- For the AI feature, I defined the input data, required the output not to invent information, and set the minimum experience length, the timeout, the rate limit and how errors are shown.

2. Incomplete features and technical challenges

2.1 Incomplete features / limitations

- **Not deployed:** the application only runs locally.
- **No login / permissions and no CV upload** (both are outside the scope of the assignment).

- **A summary can have fewer than 3 lines** when the experience description contains very little information. The system caps the summary at 5 lines but does not force a minimum of 3, because forcing it would make the AI invent information.
- **A summary can become outdated:** if the experience description is edited after summarizing, the old summary is kept until "Regenerate" is clicked.
- **The summary language** follows the language of the experience description, not the language currently selected in the interface.

2.2 Technical challenges and how they were solved

Challenge	Cause	Solution
The `@google/genai` library could not be imported into the backend	The library only supports ESM, while the backend uses CommonJS (error TS1479)	Load it with a dynamic `import()` and use `resolution-mode` for the type declaration
Accent-insensitive search (typing `quy` finds `Quý`)	PostgreSQL's `unaccent` extension is blocked by a Windows security policy, and it would also have to be installed on every database	Use PostgreSQL's built-in `translate()` function with a table of Vietnamese characters, and remove accents from the search text in Node, so nothing extra needs to be installed